

Building the IoT Data Pipeline

From Encrypted MQTT Telemetry to Web-Based Observability



Lab Module: Full Stack IoT Architecture

SYSTEM ARCHITECTURE: THE DATA JOURNEY



CORE CONCEPT: SEPARATION OF CONCERNS.
Devices publish events; the Gateway handles security; the Web Layer focuses on visualisation.

Provisioning the Persistence Engine



Windows Setup

- Download x64 MSI Installer
- Run Wizard
- Select: Install as Service
- Verification: PowerShell `Get-Service MongoDB`



macOS Setup

- Terminal: `brew tap mongodb/brew`
- Terminal: `brew install mongodb-community`
- Start: `brew services start mongodb-community`
- Verification: `brew services list`

Recommendation: Always install as a background Service to ensure automatic startup on reboot.

Verifying the Database Service

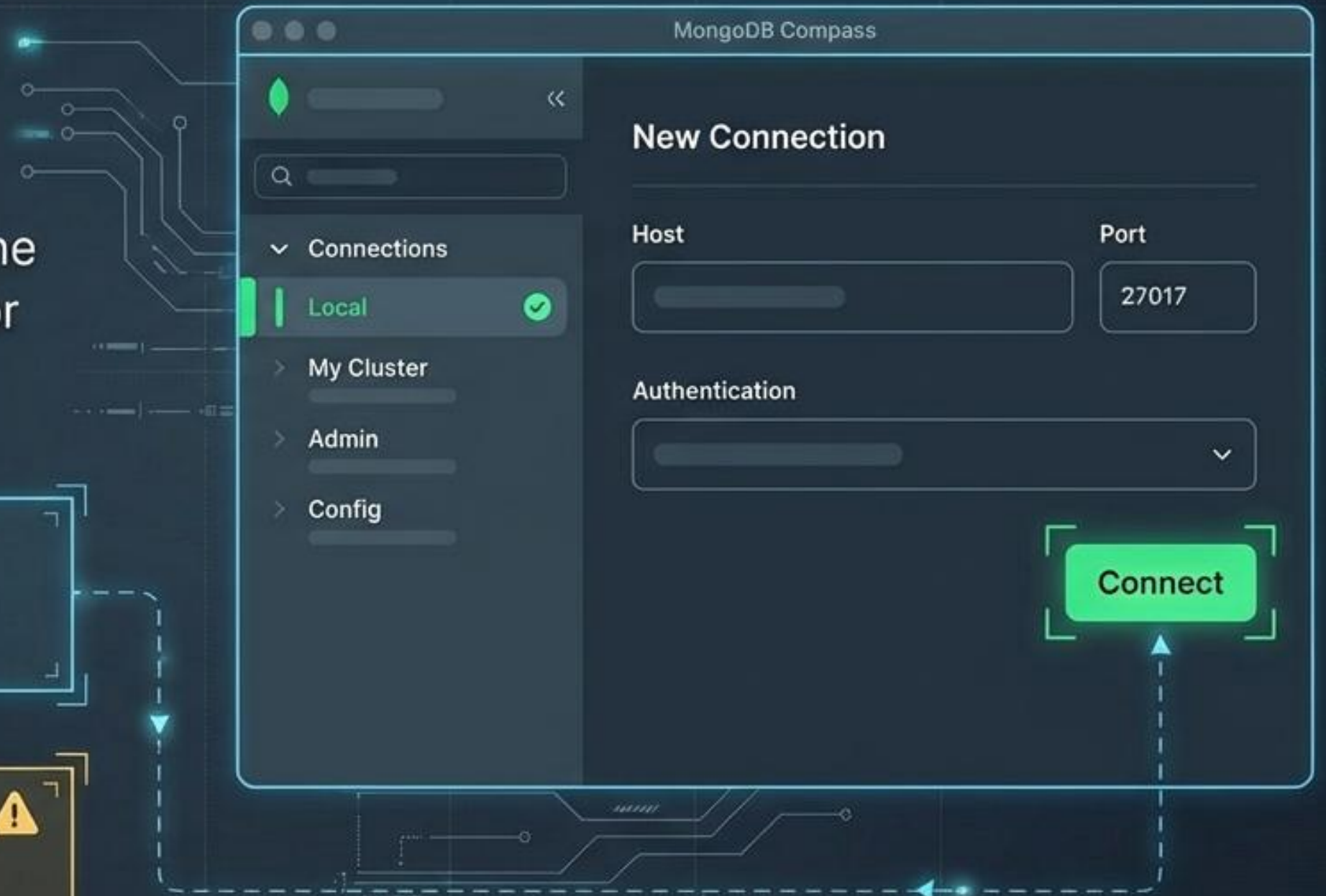
The Verification Tool

We use MongoDB Compass to visualize the state of the database. It provides a GUI for our data stores.

Connection String:

`mongodb://localhost:27017`

Note: Databases will not appear until data is written. ⚠



The 'Hello World' Connection

Terminal Window

```
user@iot-lab:~$ python3 -m venv venv
user@iot-lab:~$ source venv/bin/activate
user@iot-lab:~$ export MONGO_URL="mongodb://localhost:27017"
```

Environment activated. MONGO_URL set.

Code Editor Window - Python

mongo_test.py

```
import pymongo
import os

client = pymongo.MongoClient(os.getenv("MONGO_URL"))
db = client.setu_soil

# Test Insert
db.test_collection.insert_one({"status": "connection_active"})
print("Document inserted successfully")
```

Output: Document inserted successfully

Goal: Prove Python connectivity by inserting a dummy document before building complex logic.



The Gateway Logic

Dependencies:

- paho-mqtt
- cryptography
- pymongo



⚠ The Gateway acts as the secure bridge, stripping away encryption so the database stores clean, usable JSON data.

Gateway Configuration & Execution

Directory Tree



Action Terminal

```
user@iot-lab:~$ export MQTT_BROKER="test.mosquitto.org"
user@iot-lab:~$ export MQTT_TOPIC="setu/+/soil/+"
user@iot-lab:~$ python gateway.py
[INFO] Connected to Broker. Listening...
[DATA] Decrypted and stored in MongoDB.
```

Terminal showing environment setup and live gateway execution. ⚠

Designing the API Layer



The API Contract (Endpoints):

- /api/moisture/current (GET)
- /api/relay/status (GET)
- /api/history/30m (GET)

API Implementation Structure

Project Structure



Application Logic (app.py)

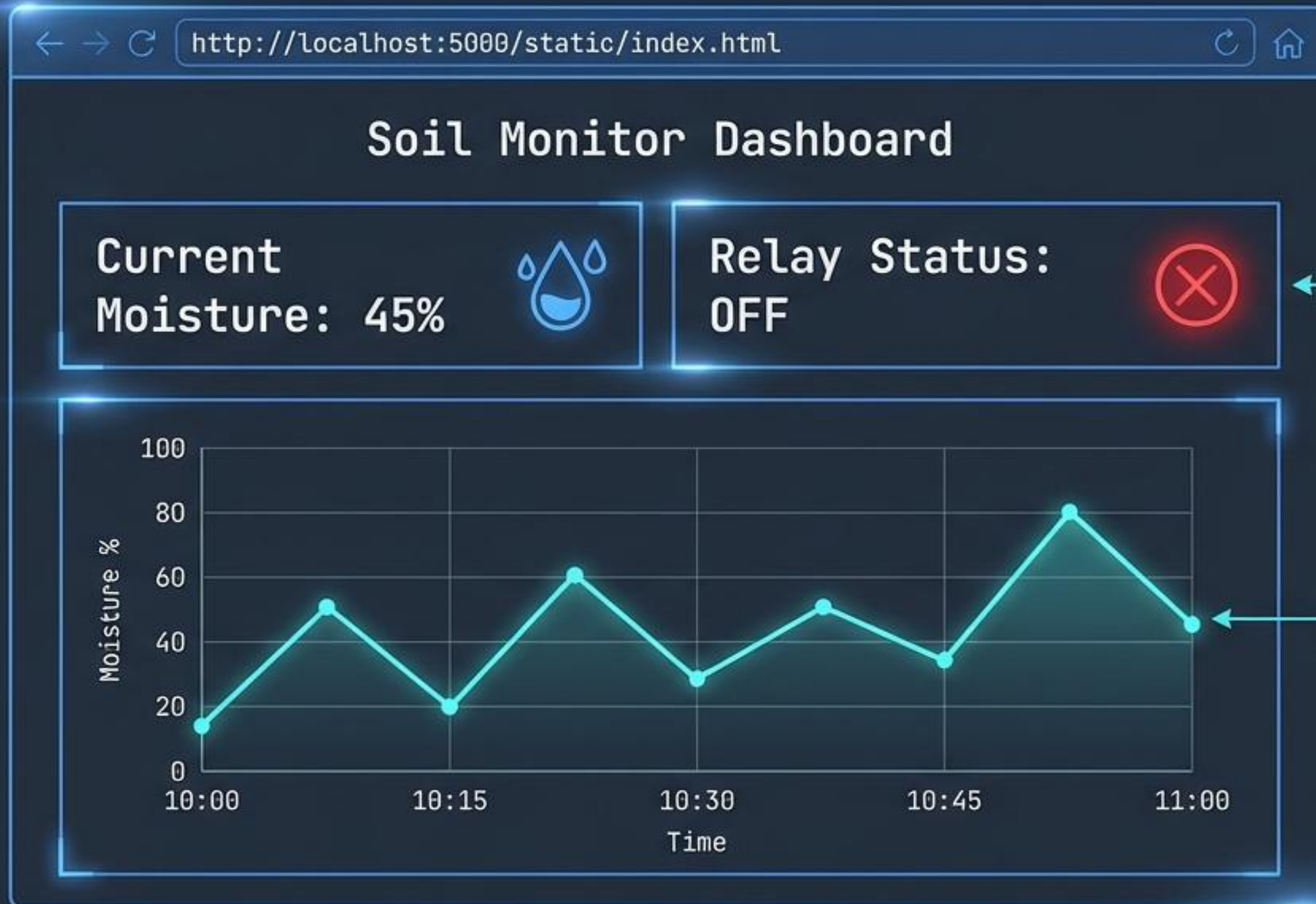
```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/api/moisture/current')
def get_current_moisture():
    # Connect to Mongo and fetch latest
    data = db.telemetry_events.find_one(
        sort=[('received_ts', -1)]
    )
    return jsonify(data)
```

⚠ The Flask app connects to the **SAME** database (**setu_soil**) populated by the Gateway.

The Frontend Dashboard



- File: `static/index.html`



- Mechanism: JavaScript `'fetch()'`



- Interval: Polling every 5 seconds

Full Stack Orchestration

Launch services in this specific order to ensure dependency availability.



Database Service

Ensure MongoDB is running (Port 27017)



Ingestion Layer

Run ``gateway.py`` in Terminal 1. (Decrypting MQTT)



API Layer

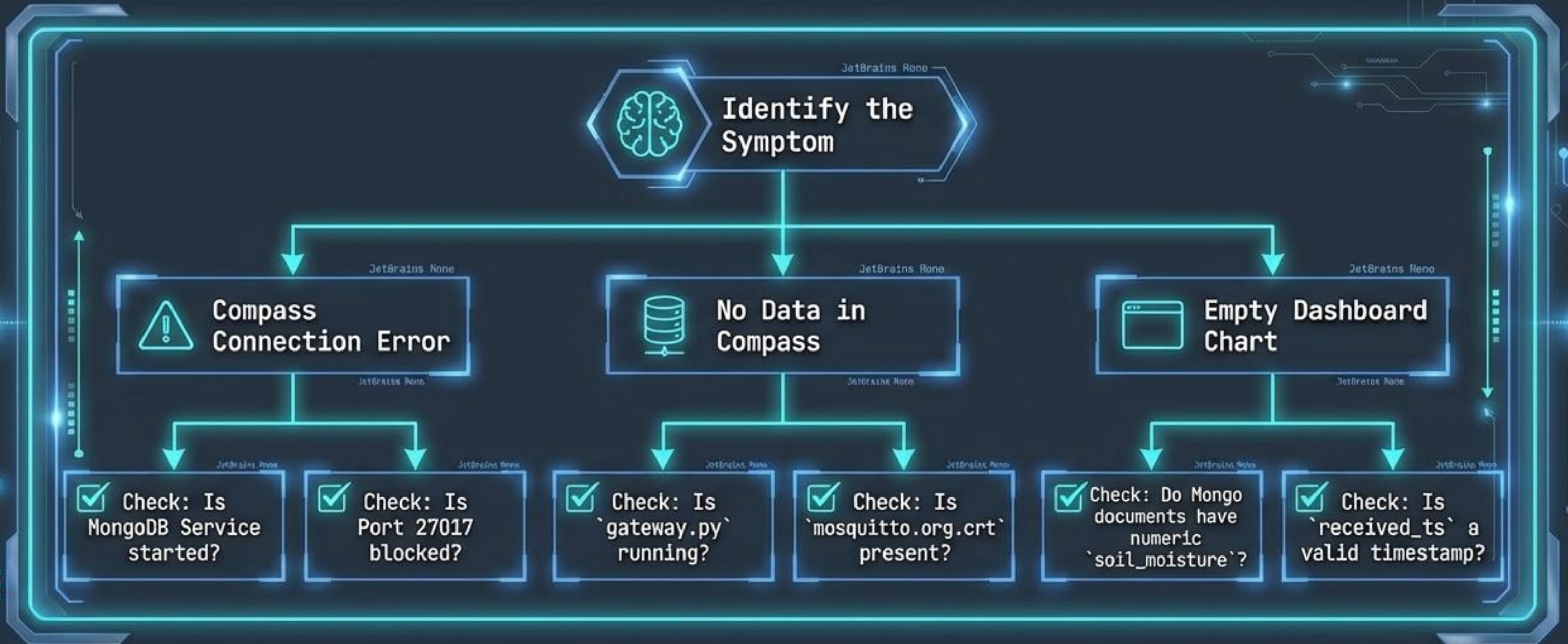
Run ``app.py`` in Terminal 2. (Serving JSON)



Presentation Layer

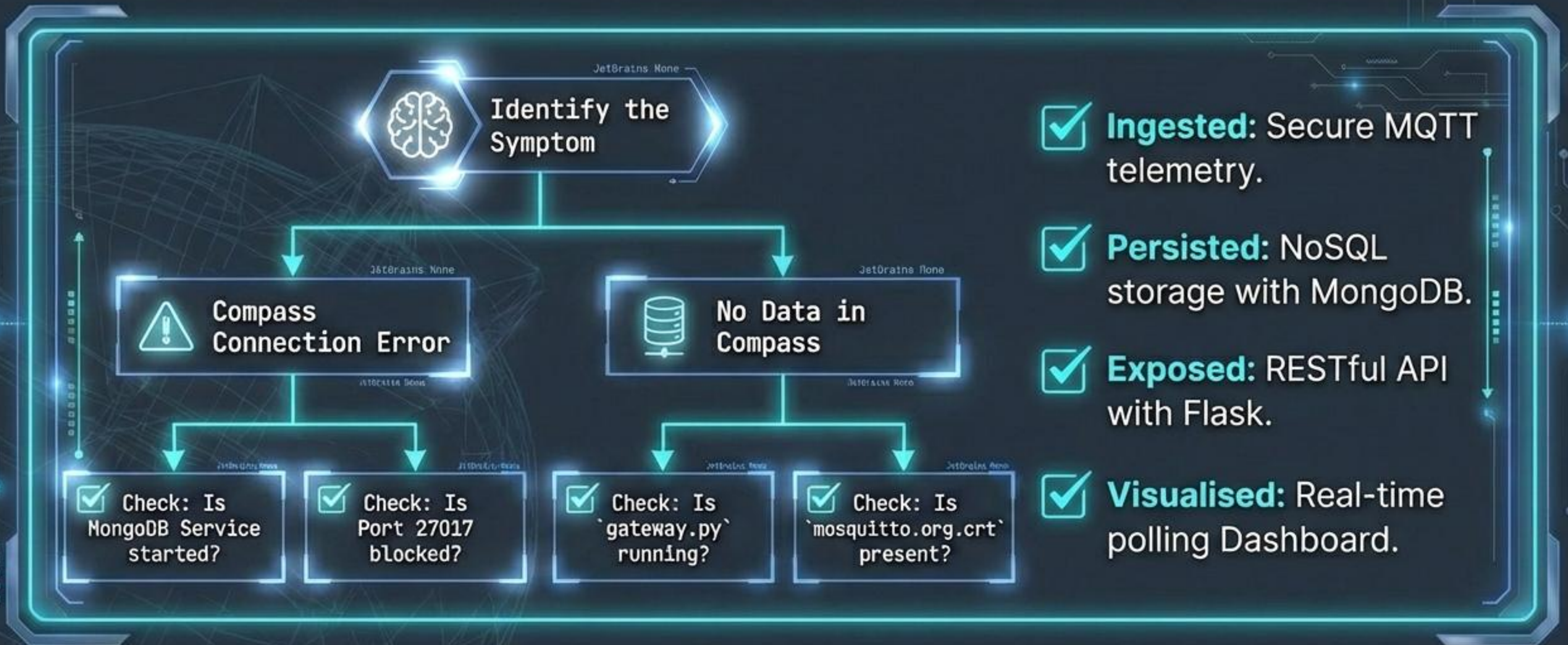
Open ``http://localhost:5000`` in Browser

Diagnostics & Troubleshooting



Golden Rule: Isolate the layer. Check Database first, then Gateway, then API.

The Complete Data Journey



"You have moved beyond scripts to build a scalable, modular IoT architecture."